Tarea 1

Analisis del proyecto

Programa: Refugio Animal — gestor CRUD con Tkinter. Cuaderno: gestion_animales_tkinter.ipynb. Enunciado: Enunciado_oficial_Tarea_1.pdf. Entrega: 12 de noviembre de 2026.

Documento que resume el trabajo: organización, relación con Tema 2 (POO/estructuras) y Tema 3 (Tkinter), y respuestas a posibles cambios de refactorización.

1. Relación con el enunciado

Requisito	Cómo se cumple
Temática libre	Refugio / inventario de animales
Diccionario complejo	animales: ID -> ficha con varios campos
CRUD	Crear, leer, actualizar, borrar
Interfaz tkinter	Clase GestorAnimales + tk/ttk
Presentación / cambios	Sección de preguntas de refactorización

Dato clave: el CRUD trabaja con DICCIONARIOS (como pide el enunciado). No hay base de datos relacional. El inventario se persiste en datos/inventario.json y las especies personalizadas en datos/especies_personalizadas.json (archivos separados).

2. Idea general

Aplicación de escritorio con las cuatro operaciones CRUD. Create: crear_animal / _crear. Read: leer_animal / _seleccionar / _actualizar_tabla. Update: actualizar_animal / _actualizar. Delete: borrar_animal / _borrar.

Frase de apertura recomendada: «Es una aplicación CRUD con Tkinter sobre diccionarios persistidos en JSON, que separa datos/validación de la lógica de interfaz (clase GestorAnimales).»

3. Mapa con el temario

3.1 Tema 2 — POO y estructuras

El enunciado pide diccionarios, no clases Animal. Aun así se vinculan ideas del Tema 2: identidad por ID; funciones CRUD como comportamiento separado de la GUI; validar_datos como contrato con ValueError; evolución posible hacia clases Animal/Perro/Gato/Loro del proyecto poo_animales.

3.2 Tema 3 — Interfaces gráficas

- Gestores de geometría: pack en bloques; grid dentro de LabelFrame.
- Variables de control: StringVar en Entry/Combobox.
- Frame/LabelFrame: formulario, filtros y zona de tabla.
- Button/Entry: acciones CRUD y campos de texto.
- Eventos: KeyRelease, ComboboxSelected, TreeviewSelect.
- Cajas de diálogo: showerror, showinfo, askyesno.
- TTK, Combobox, Text y Treeview (tabla con iid = ID).

4. Datos iniciales (diccionario)

ESPECIES = (Perro, Gato, Loro) alimenta los Combobox. animales usa un ID numérico como clave y un

diccionario como valor. Ventaja frente a una lista: acceso O(1) por ID, actualizar/borrar sin recorrer, y el ID actúa como clave primaria.

5. Funciones CRUD puras

validar_datos limpia con strip, obliga nombre/especie/color, valida edad 0-30 y peso finito > 0; si falla lanza ValueError. crear/leer/actualizar/borrar reciben el inventario como argumento. leer_animal usa copy(). crear_animal calcula max(inventario, default=0)+1. Punto fuerte: se pueden probar sin abrir ninguna ventana.

6. Prueba rápida

inventario_prueba = {} independiente. Flujo: Crear -> Leer -> Actualizar (edad 2 a 3) -> Borrar (quedan 0). Mini prueba manual; en un proyecto real se usaría unittest/pytest.

7. Interfaz gráfica (GestorAnimales)

- Variables StringVar enlazan formulario y datos.
- Layout: pack en el padre; grid dentro de cada LabelFrame (sin mezclar en el mismo contenedor).
- Treeview: show=headings, selectmode=browse, iid=str(id).
- Eventos: búsqueda en tiempo real, filtro de especie, carga al seleccionar fila.
- Acciones: try/except ValueError -> messagebox; borrado con askyesno.
- _actualizar_tabla reconstruye filas aplicando filtros de texto y especie.

8. Ejecución

raiz = tk.Tk(); aplicacion = GestorAnimales(raiz, animales); raiz.mainloop().

9. Preguntas de refactorización (respuestas cortas)

- 1. Nueva especie Conejo: añadirla a ESPECIES; Combobox y filtro se actualizan solos.
- 2. Persistencia: ya hay inventario.json y especies_personalizadas.json; SQLite sería evolución.
- 3. Módulos: datos/, validaciones/, interfaz/, exportacion/, app.py (arranque).
- 4. Campo chip: registro + validación + Checkbutton/BooleanVar + columna Treeview + seleccionar/formulario.
- 5. POO: capa paralela con AnimalRefugio + adaptador; el CRUD sigue en diccionarios.
- 6. Nombres repetidos: no hay problema; el único es el ID.
- 7. Estadísticas: Toplevel con Counter por especie y medias; opcional Canvas/matplotlib.
- 8. ¿Por qué borrar/reinsertar la tabla? Con filtros es más simple; en inventarios pequeños es suficiente.
- 9. Peso con coma: replace(',', '.') antes de float.
- 10. Confirmar salida: protocol WM_DELETE_WINDOW + askyesno.

10. Consejos para explicar el proyecto

- Empieza por lo general: CRUD + separación datos/GUI.
- Nombra principios: validación con excepciones, variables de control, gestores de geometría.
- Si piden un cambio en vivo: lee, explica el plan, luego edita.
- Si algo no está: «lo haría así…» y describe el enfoque.
- Domina pack / grid / place (aquí no se usa place).

11. Checklist antes de presentar

- Ejecutar el cuaderno completo (datos, CRUD, prueba, GUI).
- Probar crear/editar/borrar y filtros.
- Provocar un error de validación y mostrar el messagebox.
- Señalar validar_datos y el Treeview.
- Tener listas las respuestas de especie nueva e inventario.json.
- Mostrar que especies e inventario son JSON distintos.
- Relacionar al menos 5 widgets con lecciones del Tema 3.